



KV cache optimization in RL

Sophia Yang, Elaine Hou
Aug 18th, 2026

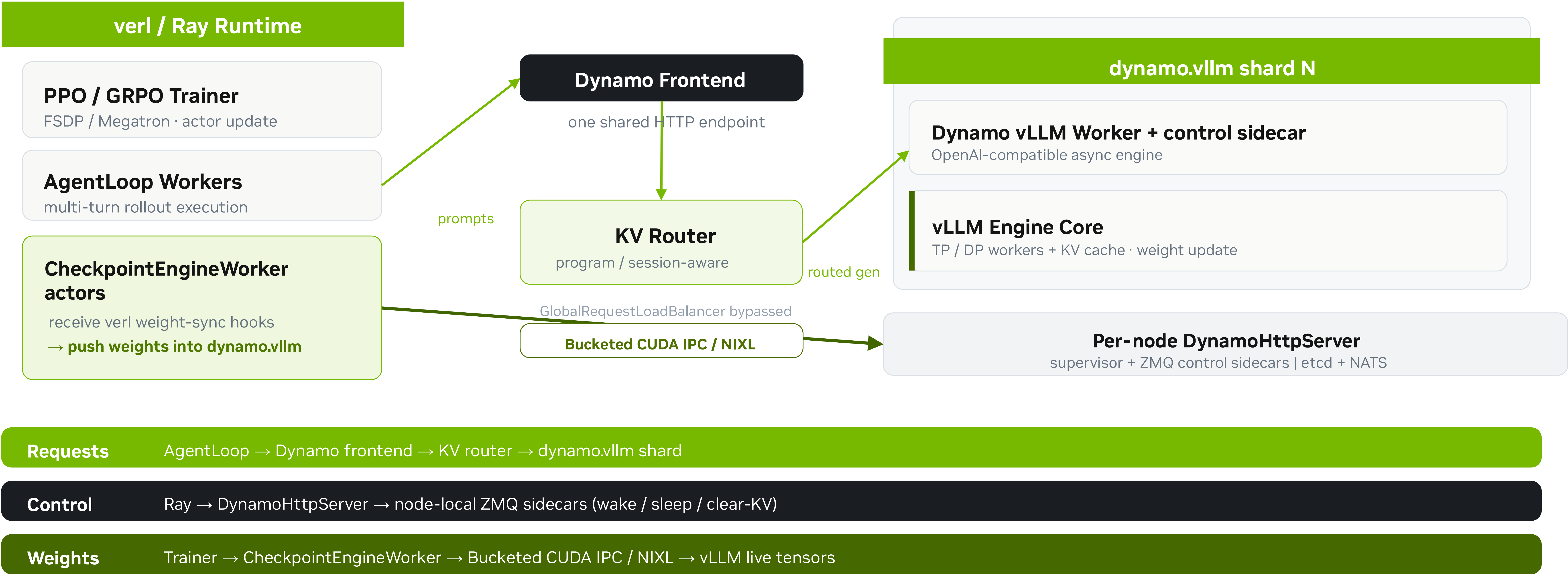


Summary

- Dynamo integration into RL framework (vllm backend done, sglang planned)
 - <https://github.com/verl-project/verl-recipe/tree/main/dynamo>
- KV aware router
 - <https://github.com/verl-project/verl-recipe/pull/110> merged
 - Retool: Qwen 3 30B MoE, rollout speedup ~7%
- Thunder agent
 - <https://github.com/verl-project/verl-recipe/pull/126> merged
 - SWE RL: 1.94× rollout throughput and 1.39× observed full-step throughput
- KV cache offload with flexkv
 - Qwen 32B, 5% speedup

Dynamo Integration into verl

Weight update rides a dedicated path alongside request routing and control



PR in verl

- Dynamo integration in verl: <https://github.com/verl-project/verl-recipe/pull/110>
- Thunder agent added in verl: <https://github.com/verl-project/verl-recipe/pull/126>
- How to use Dynamo as rollout backend:

```
export VERL_USE_EXTERNAL_MODULES=recipe.dynamo.register
python3 recipe/dynamo/main_dynamo.py \
    algorithm.adv_estimator=grpo \
    data.train_files=.../gsm8k/train.parquet \
    data.val_files=.../gsm8k/test.parquet \
    actor_rollout_ref.model.path=Owen/Owen2.5-0.5B-Instruct \
    actor_rollout_ref.rollout.name=dynamo \
    actor_rollout_ref.rollout.mode=async \
    actor_rollout_ref.rollout.engine_kwargs.dynamo.router_mode=kv \
    trainer.n_gpus_per_node=2 trainer.nnodes=1 \
    trainer.total_training_steps=2
```

PR in verl

- Dynamo integration in verl: <https://github.com/verl-project/verl-recipe/pull/110>
- Thunder agent added in verl: <https://github.com/verl-project/verl-recipe/pull/126>
- Thunder agent PR in dynamo: <https://github.com/ai-dynamo/dynamo/pull/11185> (merged)

Session Aware:

- <https://github.com/ai-dynamo/dynamo/pull/11616> (not merged, dynamo team is still considering the design)

KV Router: 综合考虑KV cache 和当前负载

RL Router

Sticky session / least loaded

Request之间

只知道：

- 哪台 Worker 当前更忙

不知道：

- 哪台 Worker 已有更多前缀缓存

Turn之间

只知道

- 哪台 Worker 已有更多前缀缓存

不知道：

- 哪台 Worker 更忙

结果：相同prompt的group内部无法复用kv cache; 某个worker过热排队

KV-aware Router

每个turn/request都打到Dynamo frontend

同时看 KV 命中 + 实时负载

总成本示例 (new tokens + workload)

Worker 1 : $0.85 + 0.30 = 1.15$

Worker 2 : $0.50 + 0.50 = 1.00$

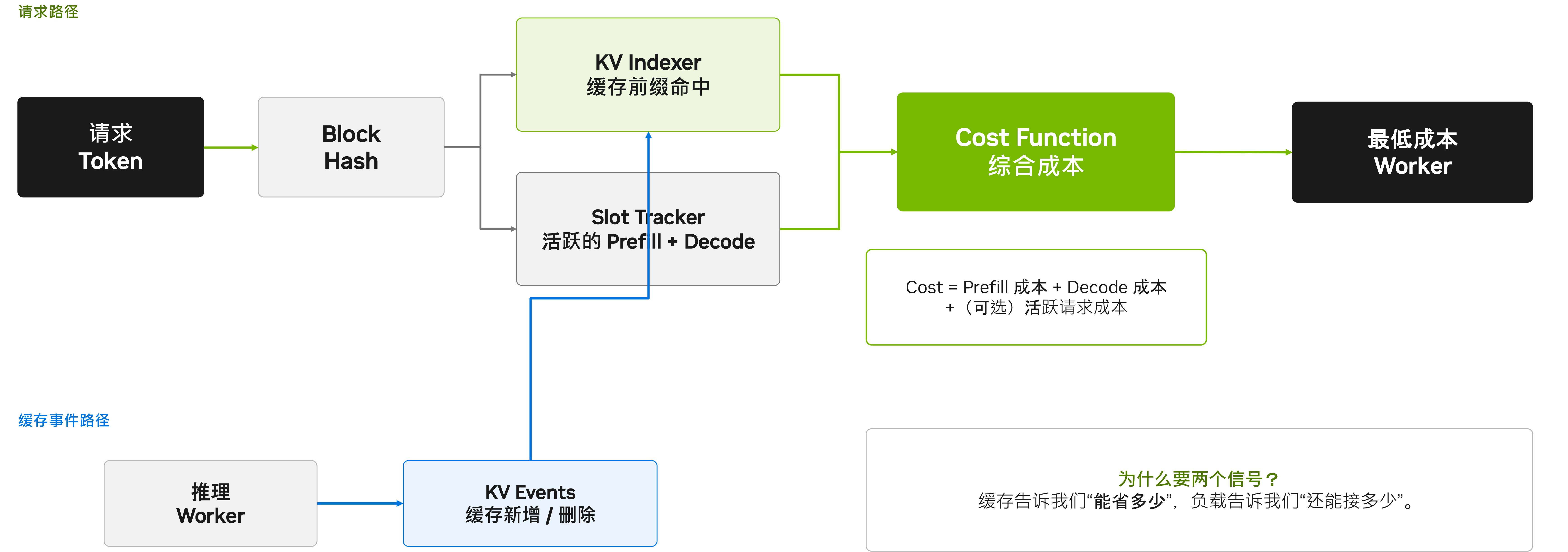
Worker 3 : $0.25 + 0.80 = 1.05$

选择 **Worker 2** : 总成本最低

Group内可以复用kv cache ; 各worker均衡负载

请求进入后，KV Router 用两类信号为 Worker 打分

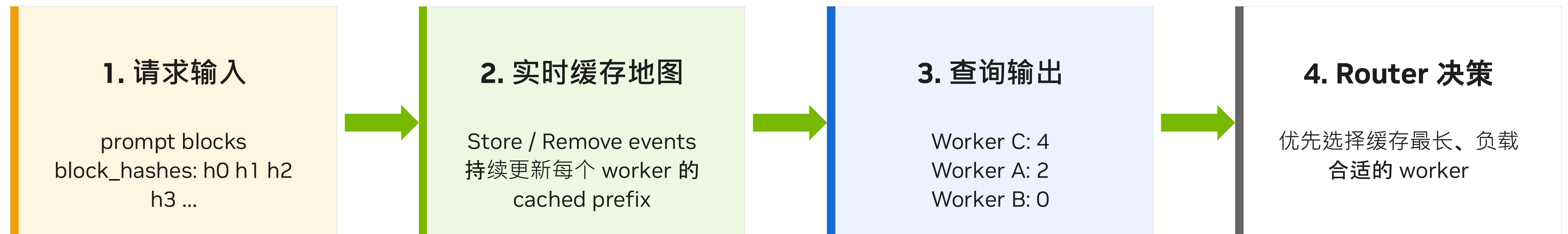
缓存命中减少重复计算；实时负载防止缓存丰富的 Worker 过载



Router 会为每个合格 Worker 打分，然后把请求发给成本最低者。

Flash Indexer 在链路里做什么

它不生成 token，也不转发请求；它回答一个路由问题：哪里已经有缓存？



为什么这很关键

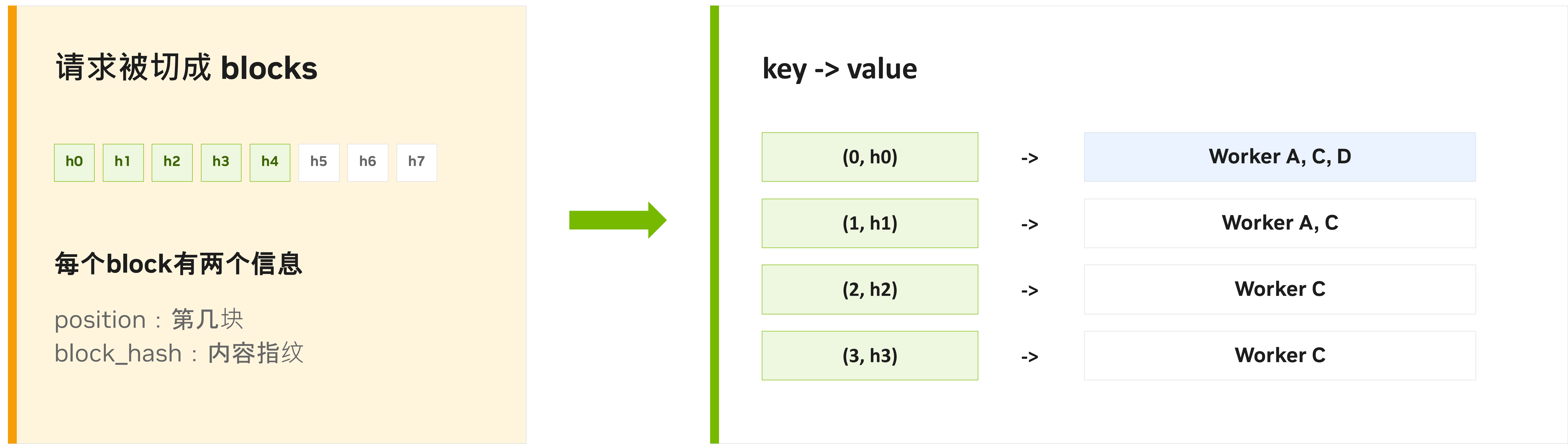
查询慢，会拖慢请求；状态旧，会错过可复用的 KV cache。

Flash Indexer 的目标

在高并发读写下，仍然快速返回“谁命中最多”，让 prefix cache 真正参与生产路由。

索引里存的是一张“位置到 worker”的地图

为了判断 prefix 是否连续，Flash Indexer 不能只看 block 内容，还要看它在第几个位置。



为什么一定要有 **position**

同一个 block_hash 可能在不同位置出现；只有位置也一致，才说明这段 prefix 真的连续。

6 代优化，其实是在解决三个瓶颈

数据结构在变，但主线始终是：少扫、少排队、少重复确认。

少扫 worker

Nested Dict -> Inverted Index

从“每个 worker 都问一遍”，变成“先按 block 找候选”。

保留上下文

Radix Tree -> Positional Index

不只知道内容相同，还知道它是否出现在同一个前缀位置。

释放并发

Rust Actor -> Concurrent Radix

读请求可以并行查；同一 worker 的写事件仍然保序。



Naive Nested Dict

第一版像一本按 worker 分册的账本：要找某个 prefix，只能先翻 W1，再翻 W2，再翻 W3。

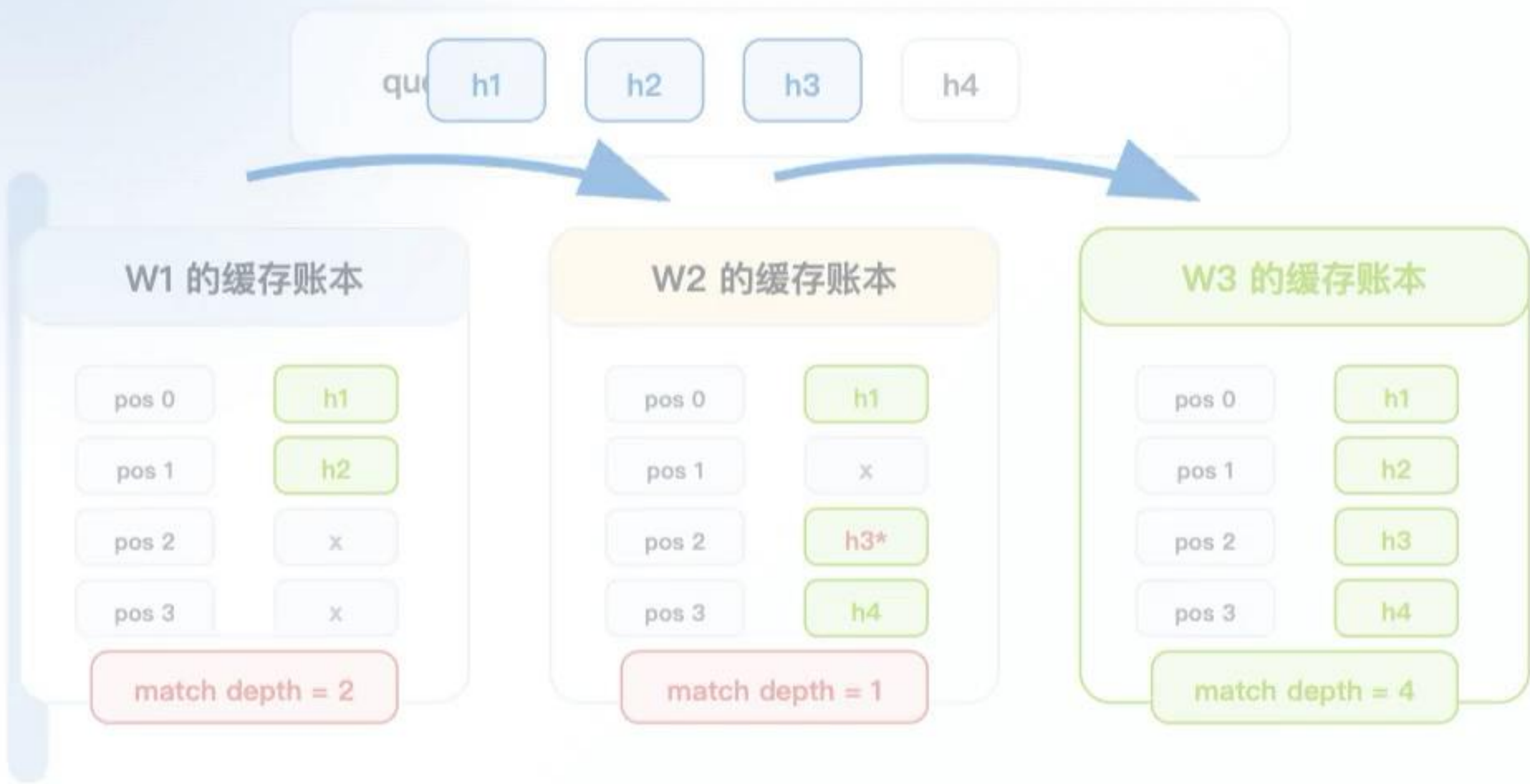
THROUGHPUT

≈385K

block ops/s

这版能工作，但查询路径很笨：每个请求都要逐个 worker 试一遍，看它的缓存能连续命中多深。

Nested Dict: 逐个 worker 翻账本



慢的原因：外层扫 worker，内层扫 prefix depth。worker 越多、prompt 越长，翻得越久。
h3* 提醒：只看 local_hash 还可能把“同内容、不同前缀位置”的 block 混淆。

为什么慢

每个请求都要扫所有 worker。worker 越多、prompt 越长，查询越像在全仓库逐柜翻找。

为什么还不够准

local_hash 只代表 block 内容。相同内容出现在不同位置时，可能被误认为 prefix 连续命中。

这一代的价值

它暴露了两个核心问题：访问模式不对，以及 local_hash 碰撞需要 prefix context。

查询成本

$O(W \times D)$

Rust Actor

第二版把账本搬进 Rust，并交给一个后台 actor 独占管理：更安全、更快，但所有读写还在排同一条队。

BOTTLENECK

单线程 ceiling

serialized reads/writes

- B1

KV cache

为什么要复用
- B2

Router question

谁命中最深
- B3

Two hashes

便宜 vs 准确
- 01

Nested Dict

逐个 worker 翻账本
- 02

Rust Actor

单队列保正确
- 03

Inverted Index

按 block 反查 worker
- 04

Radix Tree

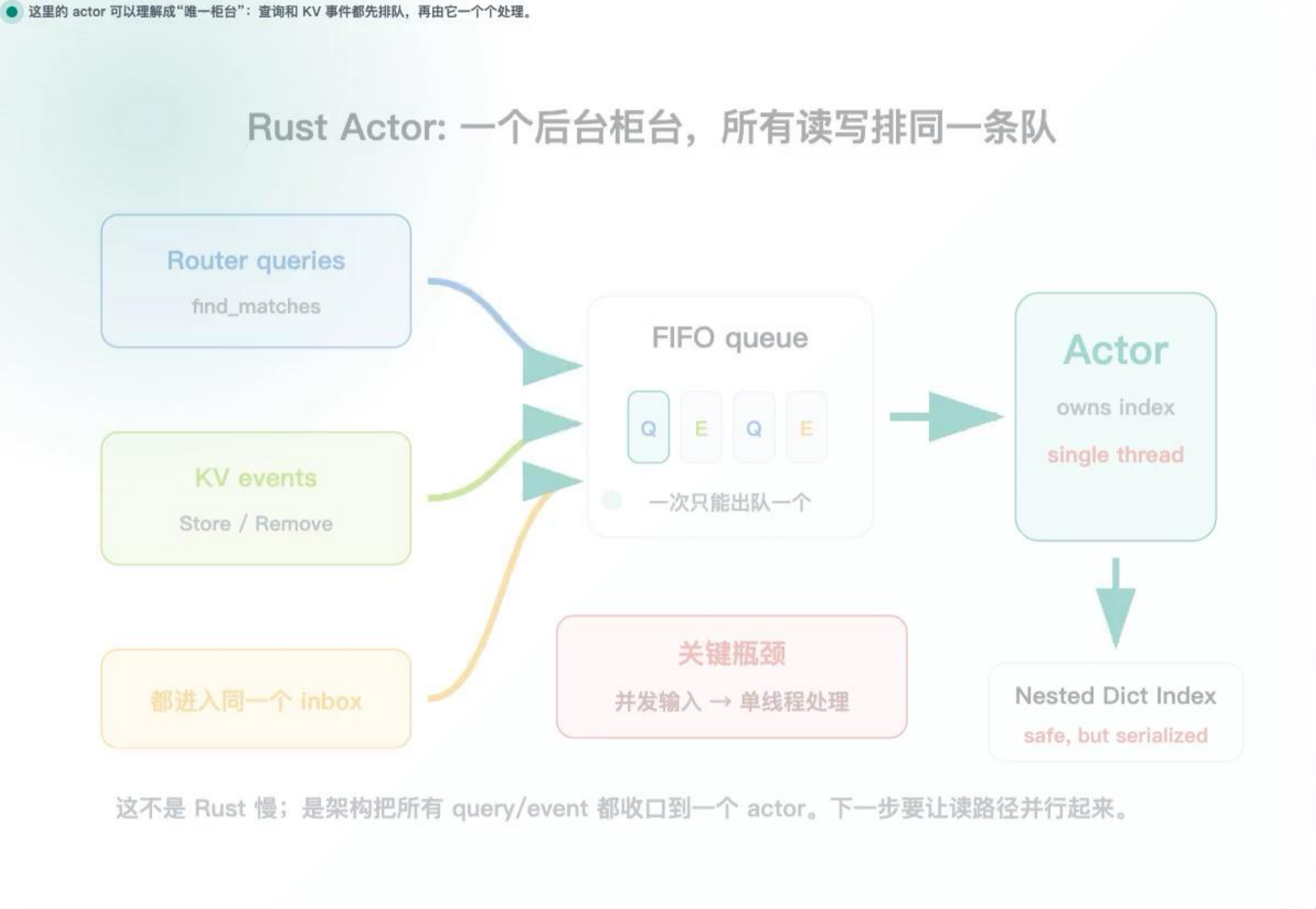
用树表达前缀
- 05

Concurrent Radix

读并行，写保序
- 06

Flash Indexer

按位置跳着查



Inverted Index

第三版把账本翻过来：不再逐个问 worker，而是按 block hash 直接反查哪些 worker 拥有它。

THROUGHPUT

≈0.8M block ops/s

- B1

KV cache

为什么要复用
- B2

Router question

谁命中最深
- B3

Two hashes

便宜 vs 准确
- 01

Nested Dict

逐个 worker 翻账本
- 02

Rust Actor

单队列保正确
- 03

Inverted Index

按 block 反查 worker
- 04

Radix Tree

用树表达前缀
- 05

Concurrent Radix

读并行，写保序
- 06

Flash Indexer

按位置跳着查

访问模式终于贴近问题：先看第一个 block 在哪些 worker，继续往后时 worker 只会逐步掉队。

Inverted Index: local_hash -> seq_hash -> workers



worker set 只会随深度 drop out

问题：collision disambiguation 和 remove 成本还没优雅解决

为什么更快

query 不再扫所有 worker 的所有 block，而是沿着请求 prefix 查每个 block 对应的 worker 集合。

还卡在哪里

local_hash 碰撞仍可能把不同 prefix 混在一起；Remove 事件如果只给 seq_hash，也不好快速删除。

这一代的突破

把查询复杂度从 $O(W \times D)$ 降到 $O(D + W)$ ：D 是请求深度，W 是最后需要打分的 worker 数。

查询成本

$O(D + W)$

Radix Tree

第四版把 prefix 变成树。相同 block 内容只有在相同父前缀下才算同一路，碰撞问题被上下文约束住。

THROUGHPUT

≈4M block ops/s

- B1

KV cache

为什么要复用
- B2

Router question

谁命中最深
- B3

Two hashes

便宜 vs 准确
- 01

Nested Dict

逐个 worker 翻账本
- 02

Rust Actor

单队列保正确
- 03

Inverted Index

按 block 反查 worker
- 04

Radix Tree

用树表达前缀
- 05

Concurrent Radix

读并行，写保序
- 06

Flash Indexer

按位置跳着查

● 树的父子关系就是 prefix context：同样的 local_hash，在不同父节点下是不同路径。

Radix Tree: prefix context scopes collisions

event lookup

worker

seq_hash

→ node

root

h1

h1

h2

h3

h2

W1,W2

W3

W4

dual-key design: local_hash for traversal, seq_hash for events

解决了什么

碰撞不再只是 hash 层面的问题，而被放进“这段内容出现在什么前缀之后”的上下文里判断。

事件怎么处理

Store/Remove 事件用 sequence_hash 快速定位节点；query 则沿 local_hash 子节点往下走。

还没解决什么

树本身更正确，但仍然在单线程 actor 之后，读写并发能力还没释放出来。

结构特点

prefix traversal

Concurrent Radix Tree

第五版让查询不用排 actor 队列：多个请求可以同时读树，写事件按 worker 分发并保持顺序。

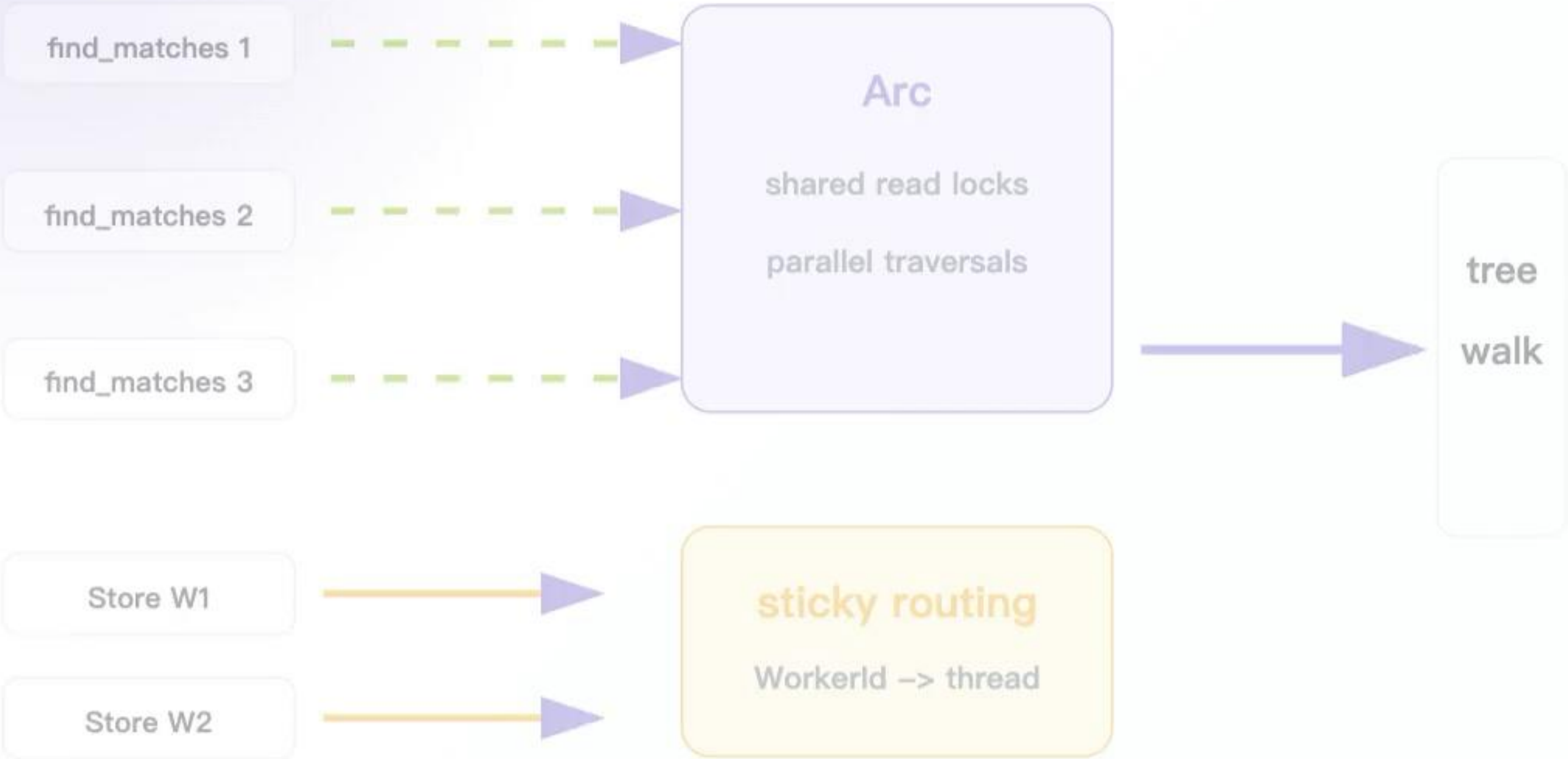
THROUGHPUT

tens of M block ops/s

B1	KV cache	为什么要复用
B2	Router question	谁命中最深
B3	Two hashes	便宜 vs 准确
01	Nested Dict	逐个 worker 翻账本
02	Rust Actor	单队列保正确
03	Inverted Index	按 block 反查 worker
04	Radix Tree	用树表达前缀
05	Concurrent Radix	读并行，写保序
06	Flash Indexer	按位置跳着查

● 这一代的关键不是换一棵树，而是让读路径绕过 actor，直接并行走树。

Concurrent Radix Tree: reads bypass actor



瓶颈移动：队列少了，pointer chasing 还在

读路径

find_matches 只拿 read lock，所以多个请求可以同时查 prefix overlap。

写路径

同一个 worker 的 Store/Remove 事件 sticky route 到同一个线程，保证顺序；不同 worker 可以并行。

新的瓶颈

并发释放了，但树 traversal 仍是 parent->child pointer chasing；长 prefix 还是要一层层走。

系统形态

parallel tree walk

Positional Indexer + Jump Search

第六版也就是 Flash Indexer：不再沿树一层层走，而是按 position 随机访问，并用 checkpoint 跳过大片共享前缀。

THROUGHPUT

≈170M block ops/s

B1

KV cache

为什么要复用

B2

Router question

谁命中最深

B3

Two hashes

便宜 vs 准确

01

Nested Dict

逐个 worker 翻账本

02

Rust Actor

单队列保正确

03

Inverted Index

按 block 反查 worker

04

Radix Tree

用树表达前缀

05

Concurrent Radix

读并行，写保序

06

Flash Indexer

按位置跳着查

这就是最后的大跳：如果第 0、64、128 个 block 都还匹配，就不必检查中间每一个 block。

Flash Indexer: position -> local_hash -> SeqEntry



D/J checkpoints instead of D pointer hops; collision path stays lazy

为什么更快

index[position][local_hash] 可以直接查某个深度，不需要从 root 一路走到那个深度。

碰撞怎么处理

大多数 slot 是 Single，直接命中；只有出现 Multi 碰撞时，才 lazy 计算 sequence_hash 消歧。

jump search

按 jump_size 跳 checkpoint。全员仍命中就跳过整段；有人掉队时，再回扫那一小段找掉队位置。

查询成本

$O(D/J + W)$

KV Aware Router in RL

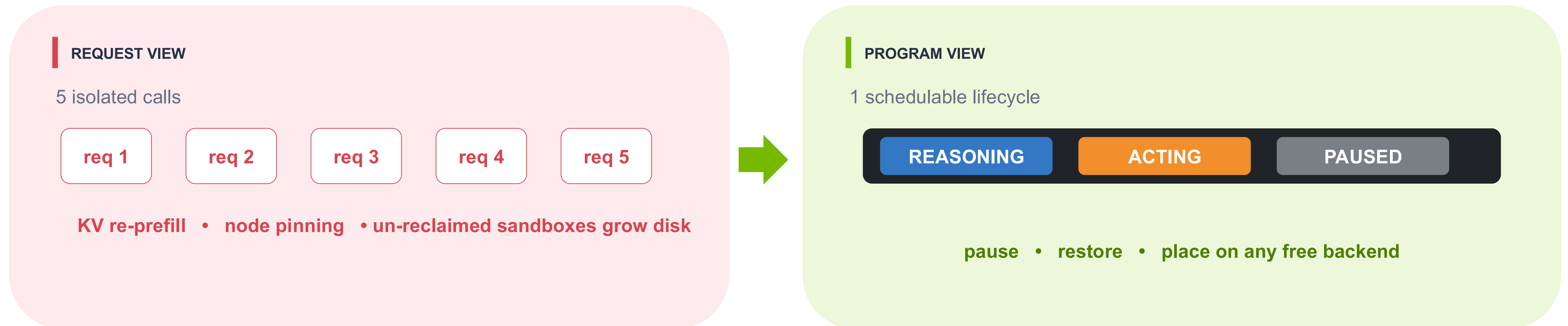
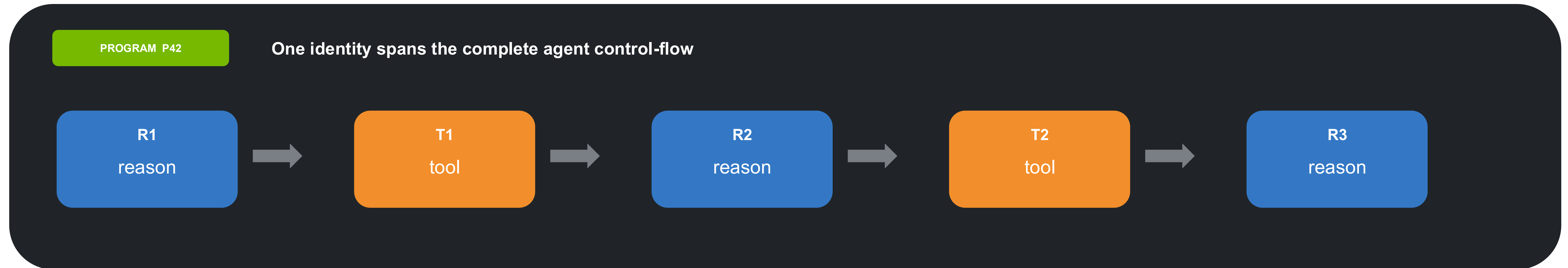
max_turns=8
max_prompt_length=2048
Prompt length mean ~300+
max_response_length=16384
n_resp_per_prompt=16
Qwen3-30B-A3B

- Dynamo (KV-aware)
- 7% faster than vLLM: 1.60 vs 1.72 ms/token
 - KV hit 89.2% vs 76.5%



Backend	ms/token	Mean response length	KV-cache hits / queries	KV-cache hit rate
Dynamo KV (stream-interval=100)	1.5956	876.1	2,248,368 / 2,520,362	89.21%
vLLM baseline	1.7220	872.3	1,860,816 / 2,432,064	76.51%

Agentic RL Rollout Pipeline



Experiment

Retool

max_turns=8

max_prompt_length=2048

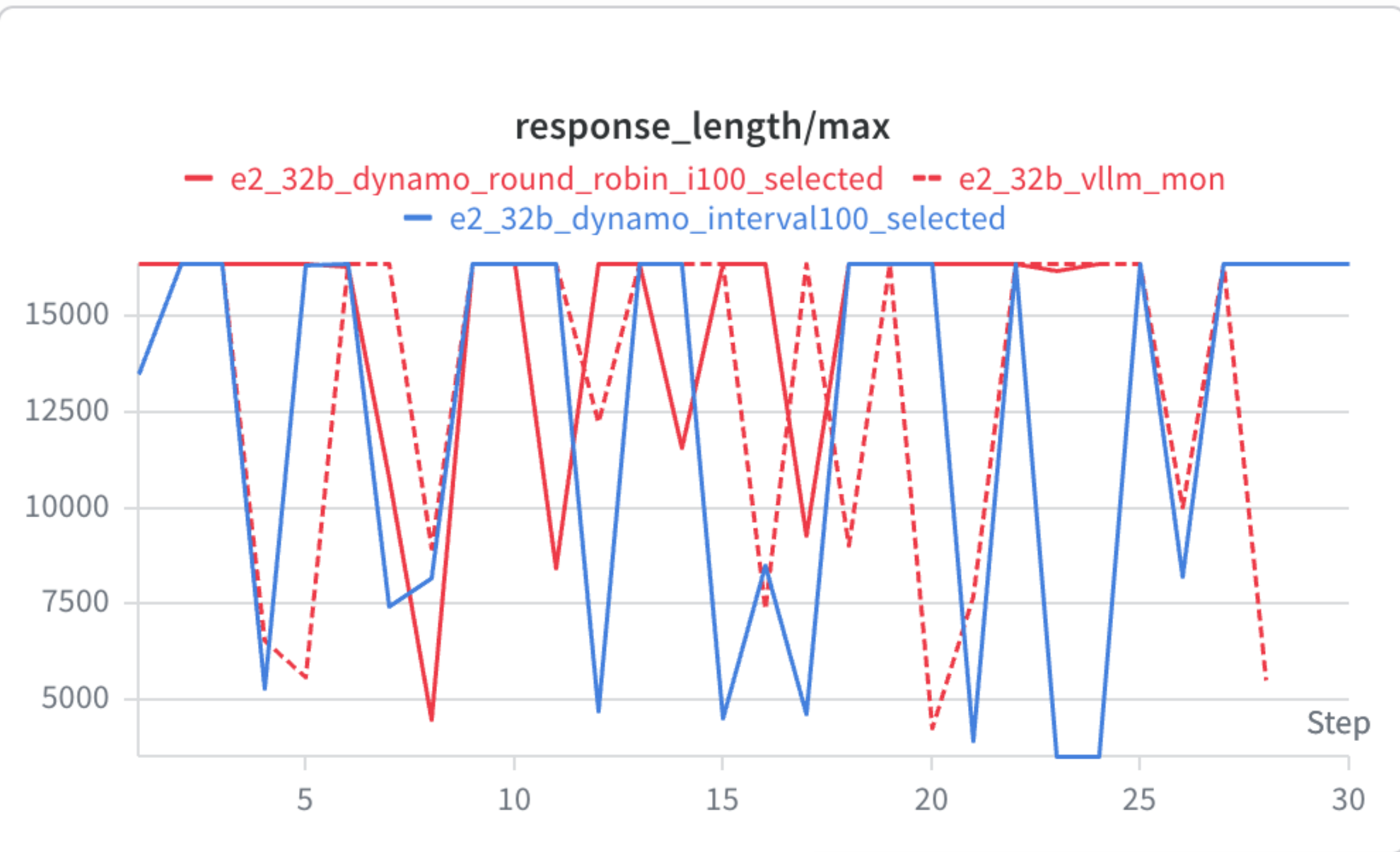
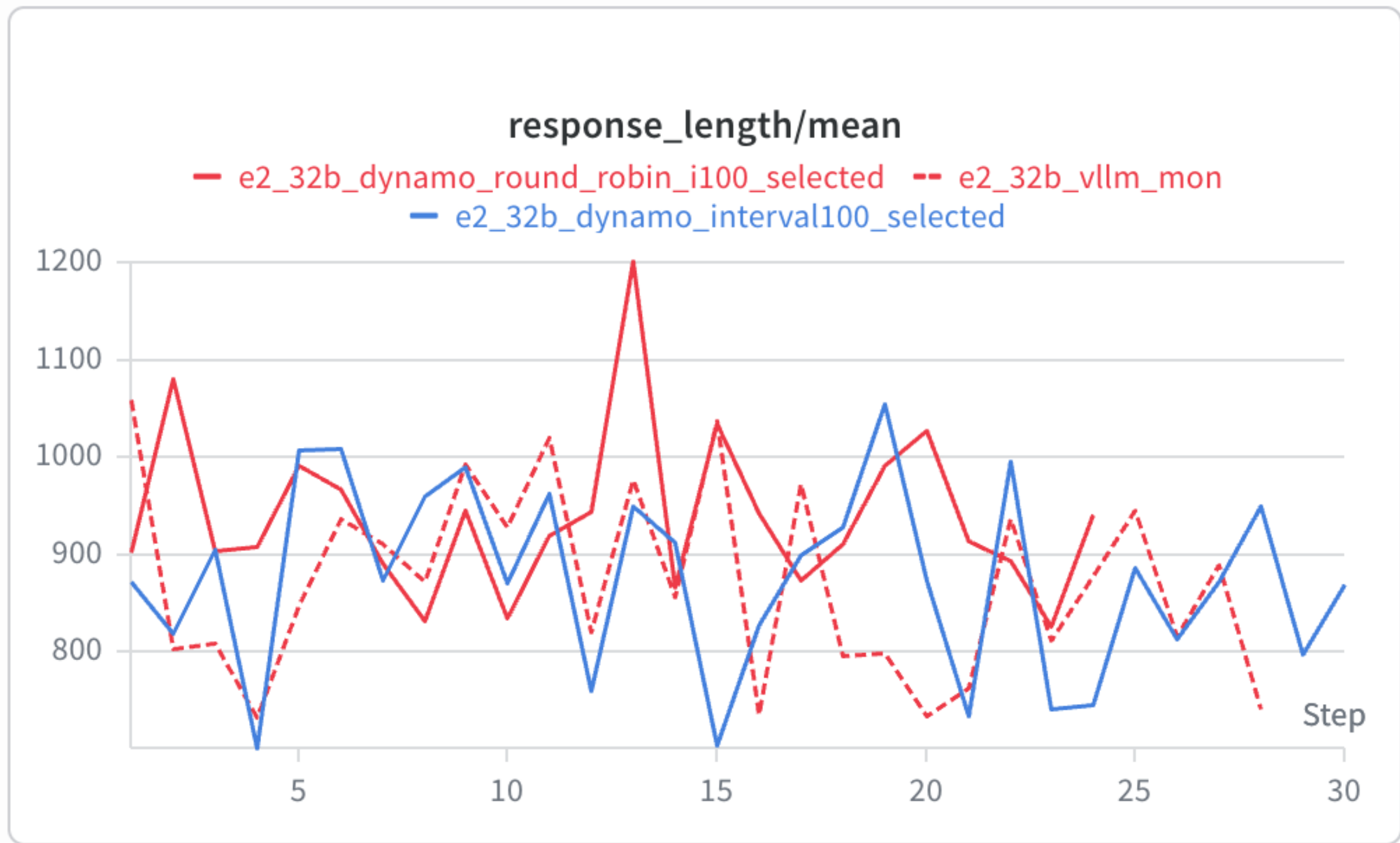
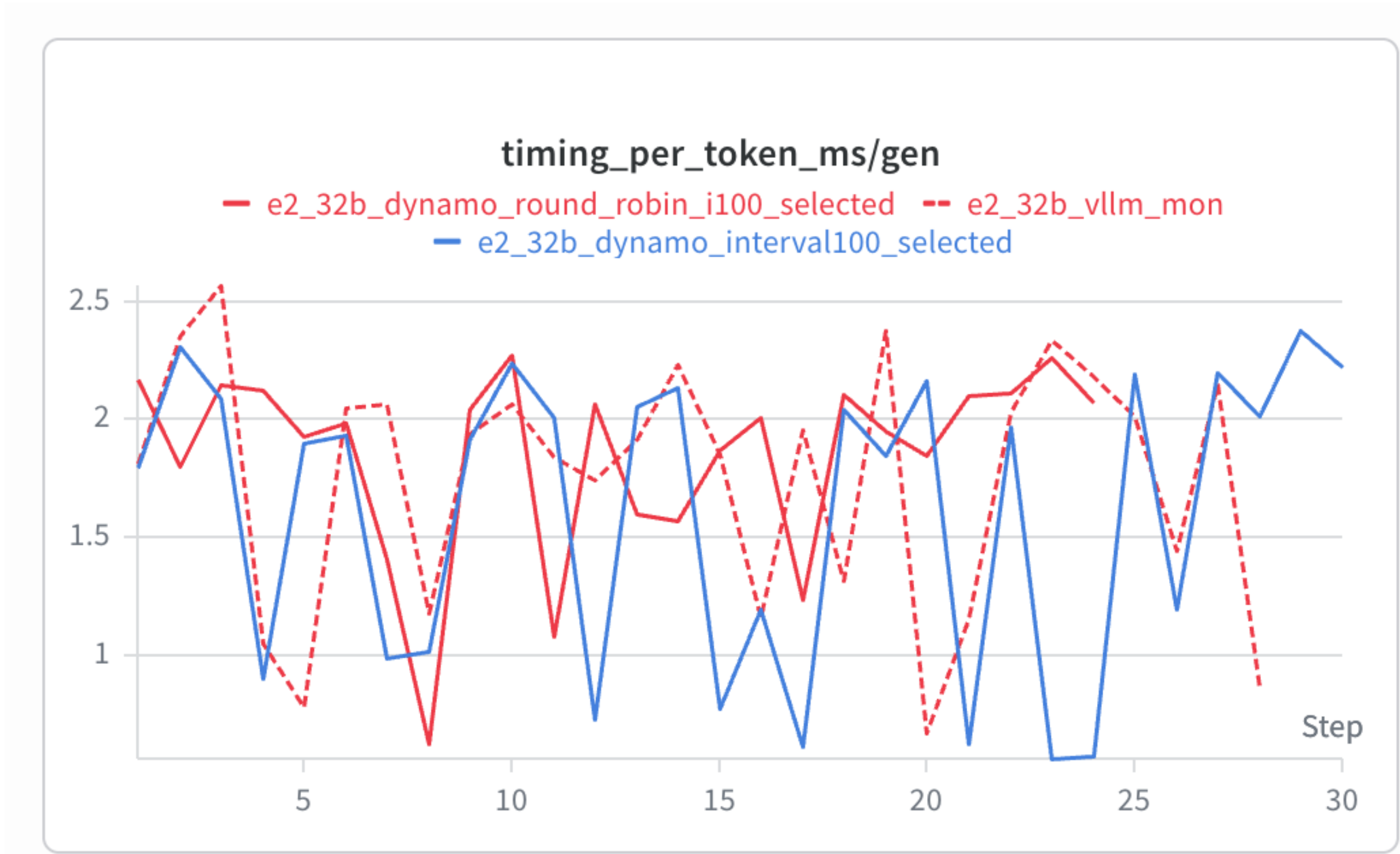
Prompt length mean ~300+

max_response_length=16384

n_resp_per_prompt=16

Qwen3-30B-A3B

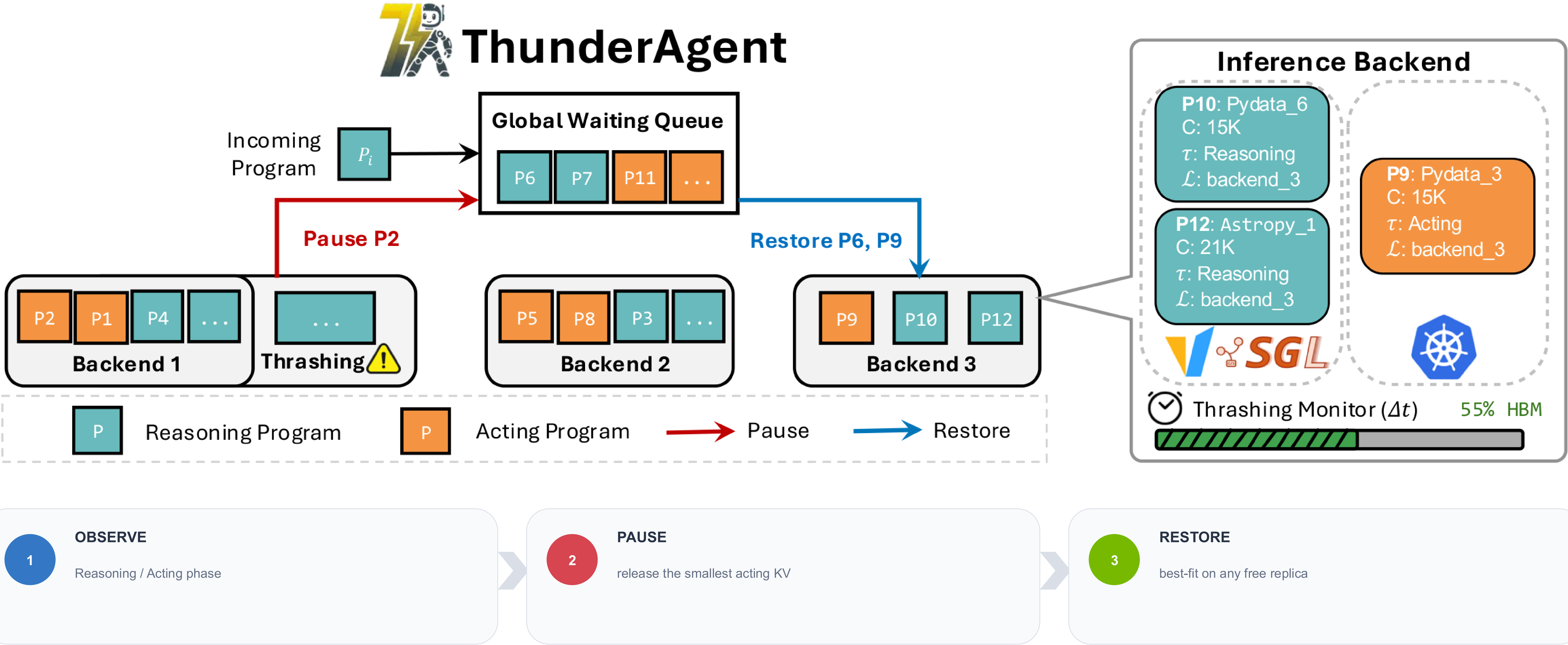
Dynamo (KV-aware) ~7% faster than vLLM: 1.60 vs 1.72 ms/token · KV hit 89.2% vs 76.5%



Backend	ms/token	Mean response length	KV-cache hits / queries	KV-cache hit rate
Dynamo KV (stream-interval=100)	1.5956	876.1	2,248,368 / 2,520,362	89.21%
vLLM baseline	1.7220	872.3	1,860,816 / 2,432,064	76.51%

ThunderAgent in one picture

Schedule the whole agent program — not each API call in isolation · arXiv:2602.13692



为什么单次 LLM Request 不是正确的调度单位

问题

Agent 的真实负载是一条长期存活、在 GPU 与外部工具之间往返的程序。



持久状态 1 • GPU KV Cache

下一轮上下文以前一轮为前缀，本应接近完全复用；工具等待时仍有未来价值。

持久状态 2 • Tool Environment

Docker、端口、依赖和仓库必须跨步骤保持一致，直到程序明确结束。

请求级视角

只看到一次 LLM 或工具调用，无法区分“等工具”和“已结束”。

程序级视角

看见身份、阶段、缓存足迹、放置位置与完整生命周期。

核心变化：调度单位从“独立请求”升级为“端到端 Agent 程序”。

请求级系统的三种浪费：重算、碎片与泄漏

问题

1 • KV Cache Thrashing

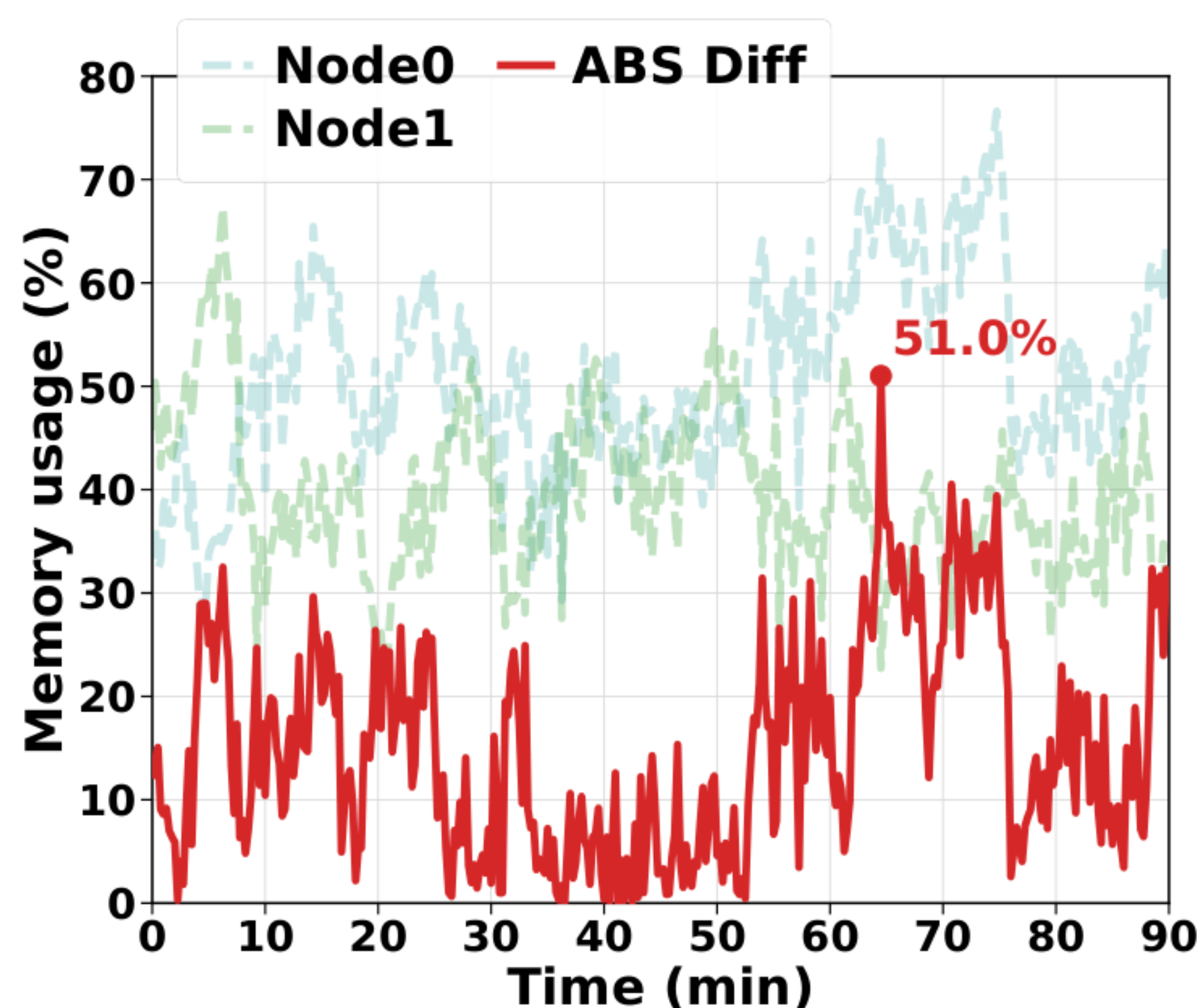
工具等待时缓存被驱逐；返回后重算完整历史。
端到端延迟最高增加 7.14×。

2 • 跨节点显存不均衡

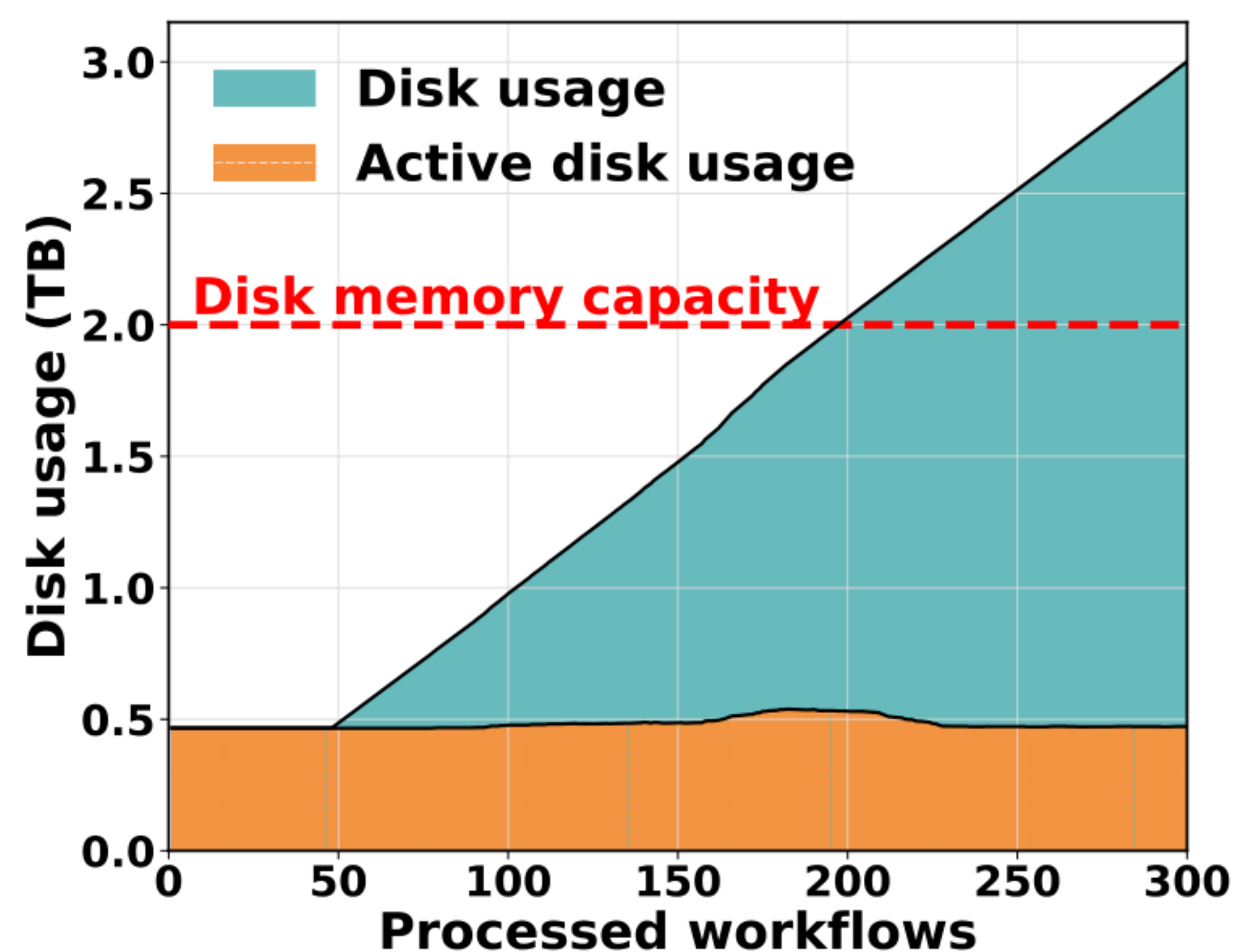
会话亲和固定节点；90 分钟中 37+ 分钟差异 >20%，峰值 51%。

3 • 工具生命周期失配

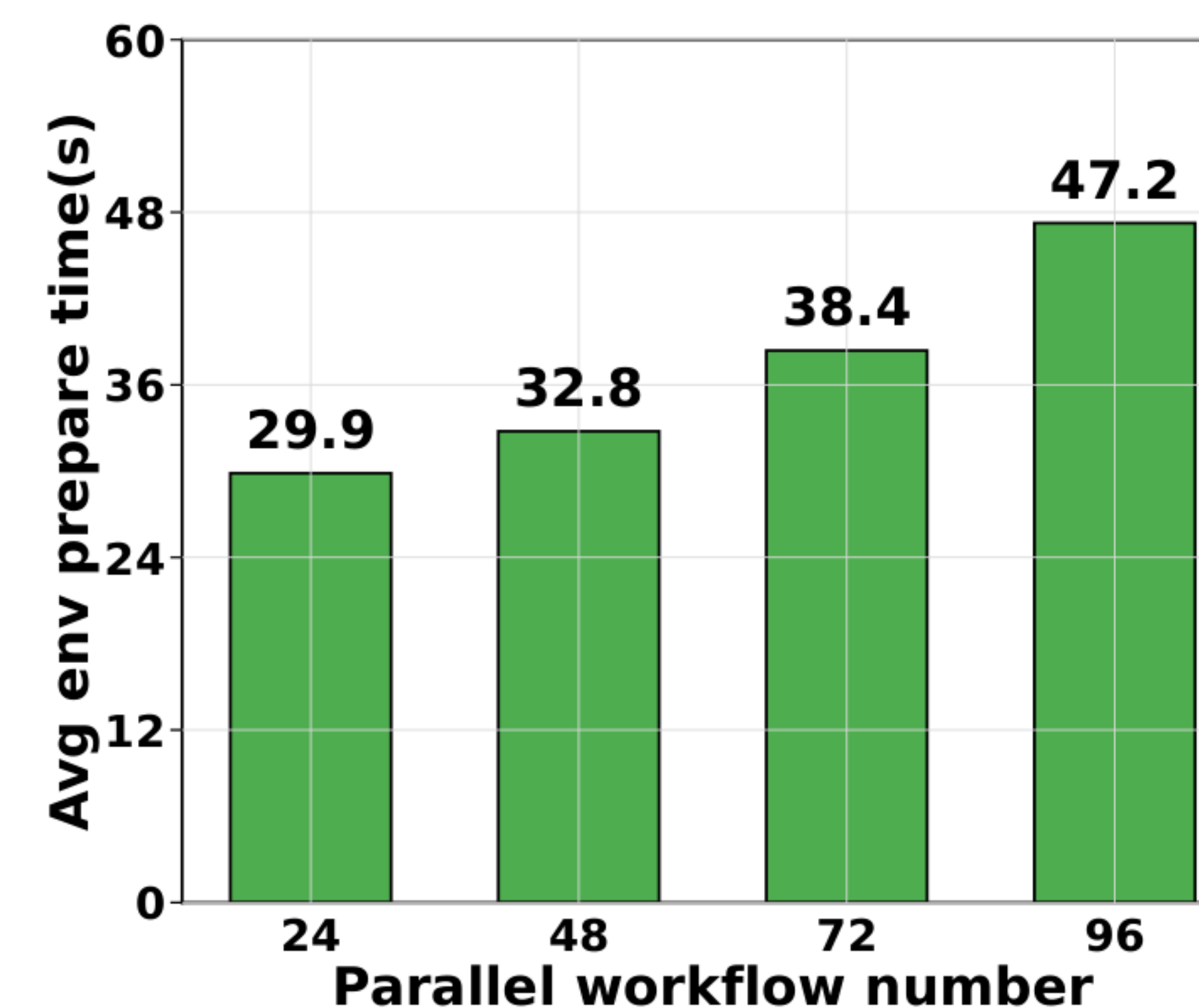
沙箱与端口不回收；环境准备时间随并发从 29.9s 增至 47.2s。



(a) Memory imbalance in rollout



(b) Runtime disk usage for tool envs.



(c) Tool env. preparation time

三个症状分别对应 Cost_recompute、Cost_unused 与工具侧生命周期开销。

程序抽象：六个字段把端到端语义暴露给运行时

抽象

$$P = (ID, c, T, L, \tau, s)$$

ID 全局程序身份

c token 数 / KV 足迹

T 工具环境集合

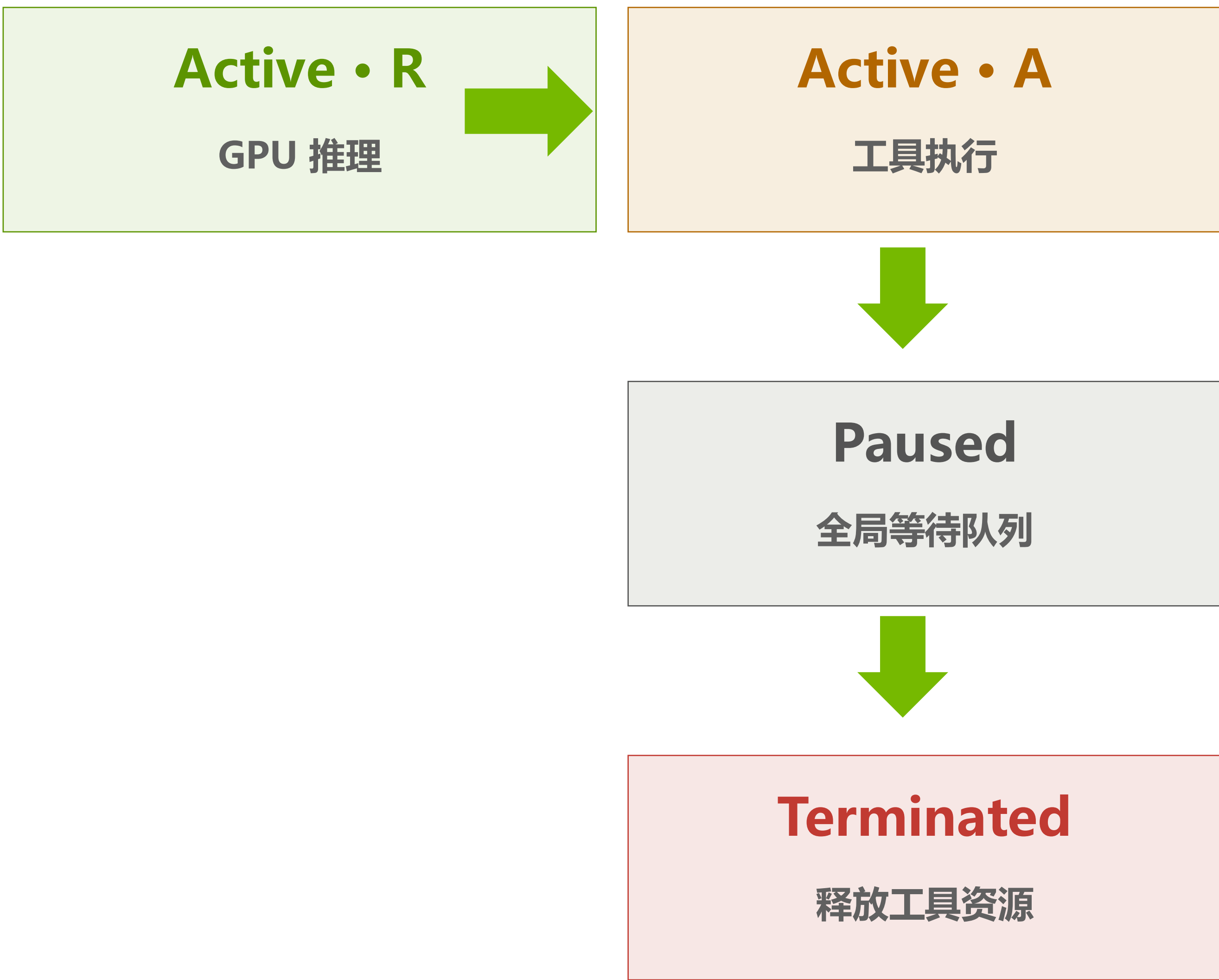
L 后端放置位置

τ 推理 / 行动

s 活跃 / 暂停 / 终止

program_id 不只是路由标签：
它让运行时聚合每条工作流的阶段、token、工具与终止信号。

同一程序跨推理、工具与暂停状态持续存在



OpenAI 风格端点封装后端，不把策略绑定到 vLLM / SGLang。

程序抽象是后续 GPU 调度与工具生命周期管理共享的“系统真相”。

成本模型：优化的不是命中率，而是“有用工作之外的 STP”

模型

$$\text{Cost_total} \approx \text{Cost_decode} + \text{Cost_prefill} + \text{Cost_recompute} + \text{Cost_unused} + \text{Cost_caching}$$

Decode

有效工作

Prefill

增量有效工作

Recompute

抖动后的历史重算

Unused

跨节点空闲容量

Caching

等工具时闲置 HBM

绿色 = 生产性 STP

暖色 = 非生产性 STP

核心权衡：缓存闲置成本 ↔ 恢复时重算成本

保留 KV
短工具调用划算



驱逐 KV
长尾等待划算

关键解读

KV 命中率是手段，不是目标。
命中率更高也可能因 HBM 闲置而吞吐更低。

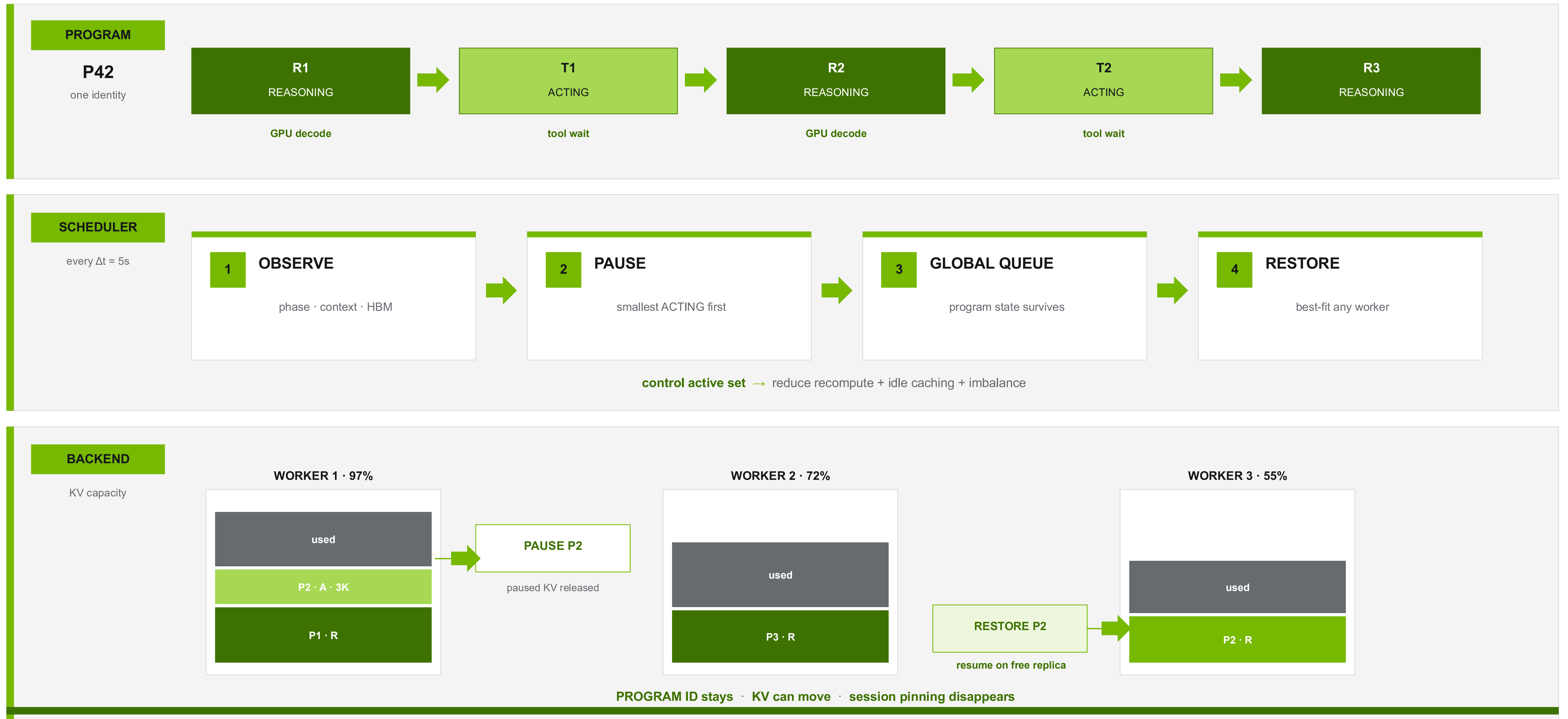
STP 定义： $\text{Cost}_x = \int M_x(t)dt$ ；实现中以 KV token 数近似内存足迹。

ThunderAgent 的目标是最小化 $\text{Recompute} + \text{Unused} + \text{Caching}$ ，而非盲目固定 KV。

$$\text{Cost_total} \approx \text{Cost_decode} + \text{Cost_prefill} + \text{Cost_recompute} + \text{Cost_unused} + \text{Cost_caching}$$

项目	STP 表示什么	性质
Decode	解码期间占用的 KV 显存 × 时间	有效工作
Prefill	增量预填充期间占用的显存 × 时间	有效工作
Recompute	KV 被驱逐后，重算历史上下文占用的显存 × 时间	浪费
Caching	Agent 等待工具时，闲置 KV 仍占用的显存 × 时间	机会成本
Unused	因节点负载不均而空着、无法被其他程序利用的 KV 容量 × 时间	机会成本

ThunderAgent: Program-Aware Scheduling



ThunderAgent Algorithm

抽象:整条 agent 轨迹 = **Program** $\langle c$: context tokens, τ : REASONING/ACTING, s : Active/Paused $\rangle$,以 program 为单位、每 $\Delta t=5s$ 逐 worker 检测
 $util = used_w / C_w$,ACTING 即时暂停、REASONING 推迟到 tool boundary,替代盲目逐请求驱逐

Pause 选谁 $S_{pause} = 1/c + 1\{\tau=ACTING\}$

ACTING 严格优先(指示函数 = 硬闸门):实现中只有 ACTING 会被真 pause;
命中的 REASONING 仅打 marked_for_pause,推迟到它下一次进入
ACTING 才落地

同相位内最短 c 优先:重算代价 $\propto c^2$ (重 prefill 全上下文), $\min \sum c^2$ s.t. $\sum c \geq \Delta C \rightarrow$ 驱逐最小者最优

Resume 选谁 $S_{restore} = 1/c + 1\{\tau=REASONING\}$

三档而非两档: $group_key = REASONING(step > 1) < 新程序(step \leq 1) < ACTING$,组内 c 升序 —— 该序只决定准入集合(贪心背包 $\sum(c+buffer) \leq$ 各 worker 剩余容量之和)

准入后按 c 降序 BFD 装箱:大者先放入剩余容量最大的 worker(与准入的升序相反);paused 的 KV 已驱逐、恢复与节点无关 $\rightarrow$ 全局队列消除跨节点失衡

时间衰减 $f(t) = 2^{(-t_idle/\tau)}$, $\tau = 1s$

⚠ 论文与 **Dynamo** 实现不一致:Eq.(7) 让 ACTING 的 c 在容量检查里乘 $f(t)$;dynamo 只在挑放置 worker 时用衰减值,pause / resume 的阈值判定用未衰减 c(acting_token_weight 默认 1.0)

设计意图是在 Cost_caching(占着不用)与 Cost_recompute(驱逐重算)间自动折中;工具时延 memoryless 时,指数衰减是唯一可行形式

控制回路 + 滞回 watermark 三段带

$util \geq \lambda_max(0.95) \rightarrow$ 依 S_{pause} 逐个暂停,直到 $\leq target(0.80)$

$0.80 \leq util < 0.95 \rightarrow$ 软降级:不暂停,只给该 worker 上的 program 打 -2.0 优先级(约 $1.5 \times \Delta t$ 有效期)

resume 只填到 0.85(= 0.95 - 滞回 0.10);新 program 排在 paused 之后(公平);等待 $> 1800s$ 强制恢复(防饿死)

优化目标(STP 代价模型):**Cost = decode + prefill | recompute + caching + unused** —— shortest-first $\downarrow$ recompute · ACTING-first pause
 $\downarrow$ caching · 全局队列 $\downarrow$ unused

ThunderAgent (arXiv:2602.13692v3, Eq.7 / 10 / 11) · Dynamo port: ai-dynamo/dynamo PR #9448 · 容量口径 PR #11185 · 代码核对 components/src/dynamo/thunderagent_router/router.py

One Step Forward: Dynamo Native Session Aware

Python TA · 进程外代理(out-of-process)



worker 选择在代理内完成 → 前缀感知放置被旁路

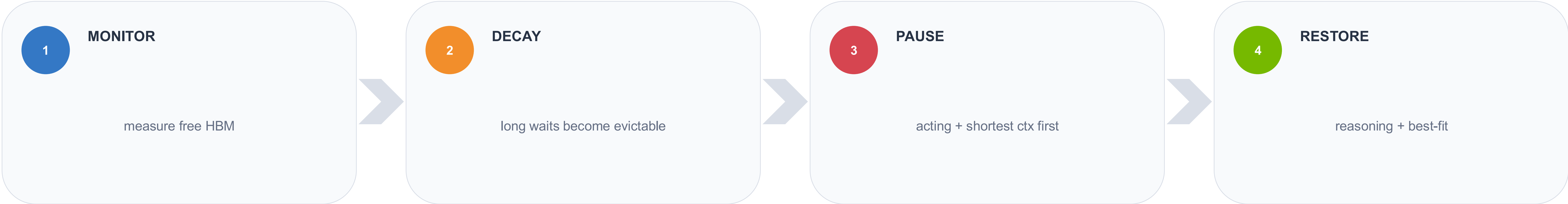
Native SessionAware · 进程内策略(in-process)



准入与前缀感知放置正交,可组合叠加

Program-aware scheduler: control the active set

Periodic monitor $\Delta t = 5$ · acting weight decays as $f(t) = 2^{-t}$



$$C_{\text{total}} < \sum_{p \in \mathcal{L}, \tau = \mathbf{R}} c_p + \sum_{q \in \mathcal{L}, \tau = \mathbf{A}} c_q \times f(t_q)$$

$$S_{\text{restore}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{R})$$

$$S_{\text{pause}}(P) = \frac{1}{c_P} + \mathbb{I}(\tau = \mathbf{A})$$

- $f(t)$ is a time-decay function designed
- **The first term** accounts for the KV cache of programs in the **Reasoning (decoding)** phase, with their full KV cache usage counted toward memory consumption.
- **The second term** accounts for the KV cache of programs in the **Acting (tool execution)** phase, where their KV cache usage is weighted by the time-decay function $f(t)$.
- C_p : context of program
- Shortest first to decrease the effort of recompute

Program-aware scheduler: control the active set

Periodic monitor $\Delta t = 5$ · acting weight decays as $f(t) = 2^{-t}$

```
248 def _program_tokens(self, program: Program, *, decayed: bool = False) -> int:
249     if program.status != ProgramStatus.ACTING:
250         return program.token_total
251     if not decayed:
252         return int(program.token_total * self._cfg.acting_token_weight)
253     tau = max(self._cfg.acting_decay_tau_seconds, 1e-3)
254     idle = (
255         max(0.0, time.monotonic() - program.acting_since)
256         if program.acting_since > 0
257         else 0.0
258     )
259     return int(program.token_total * (2.0 ** (-(idle / tau))))
```

RESUME — 选“REASONING 优先,再短优先(工具调用时间越长, 越短)”

```
- 选谁 (排序键就是优先级):
def group_key(p):
    if p.step_count <= 1:           return 1
    if p.status == REASONING:      return 0    # ← 最高优先
    return 2                        # ACTING 最低

每发起一次LLM请求, step_count加1
```

PAUSE — 选"最小的 ACTING 优先"

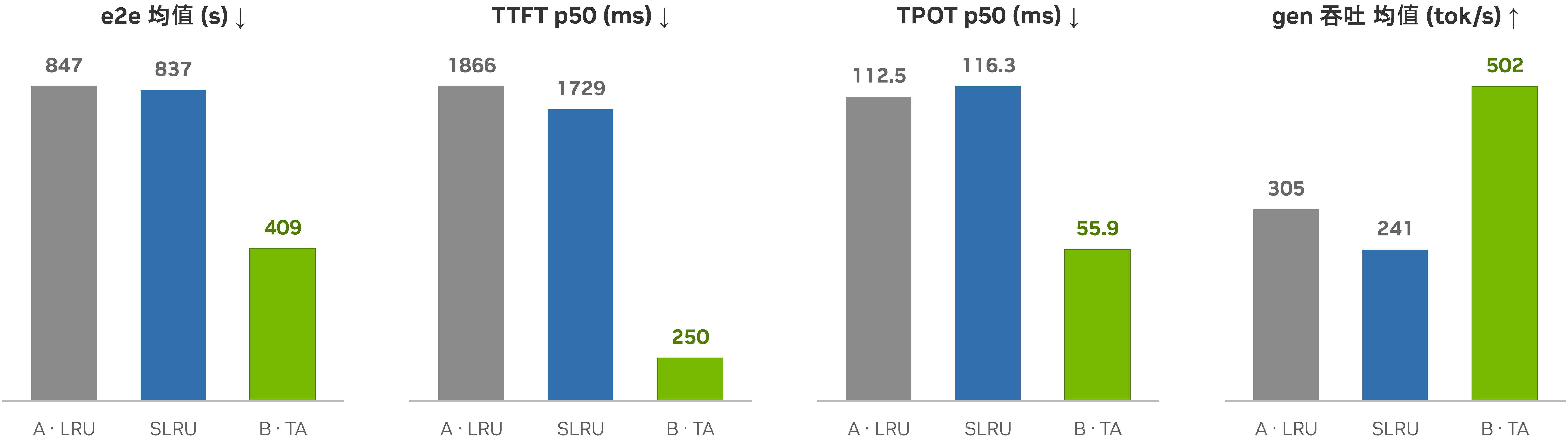
```
337 acting, reasoning = self._smallest_candidates(worker_id)
338 if acting is not None:           # ← ACTING 优先
339     if self._pause_acting_locked(acting.program_id):
340         paused_this_tick += 1
341     continue
342 if reasoning is not None: # ← 没 ACTING 才动 REASONING
343     if (
344         not reasoning.marked_for_pause
345         and reasoning.lifecycle == ProgramLifecycle.ACTIVE
346         and reasoning.status == ProgramStatus.REASONING
347     ):
348         reasoning.marked_for_pause = True
349         marked_this_tick += 1
350     continue
351     break
--- REASONING 不能中途打断,所以只是 marked_for_pause=True,等它这轮请求结束(after_request line 205)再暂停
```

组	是谁	恢复优先级
0	step≥2 且 REASONING — 跑到一半、要继续算的老程序	最高
1	steps≤1 — 刚起步的新程序	中
2	ACTING — 还在等工具、不急着算	最低

- $f(t)$ is a time-decay function designed
- 只有 ACTING(等工具)程序才衰减;REASONING(占 GPU)始终按 token_total 满权重算
- 衰减函数就是 $f(t)=2^{-(idle/\tau)}$, $\tau=acting_decay_tau_seconds$ (默认 1.0s)。等价论文的 $f(t)=2^{-t}$,只是把时间常数做成可调
- idle = 程序在 ACTING 状态待了多久 = $time.monotonic() - acting_since$ 。
- acting_since 在 program_state.py::end_request 里设置——LLM 请求一结束、程序转入 ACTING(开始等工具)就打时间戳。所以 idle 就是工具等待时长。等得越久, $2^{-(idle/\tau)}$ 越趋近 0,这条程序 token 的有效权重就越低。

Agentic Serving ThunderAgent Benchmarks

SWE-Bench Lite · GLM 5.2 · 8× NVIDIA Hopper GPUs · session concurrency = 32



时延:e2e 均值 -51% · TTFT p50 -87% · TPOT 减半 | 吞吐:均值 ×1.6

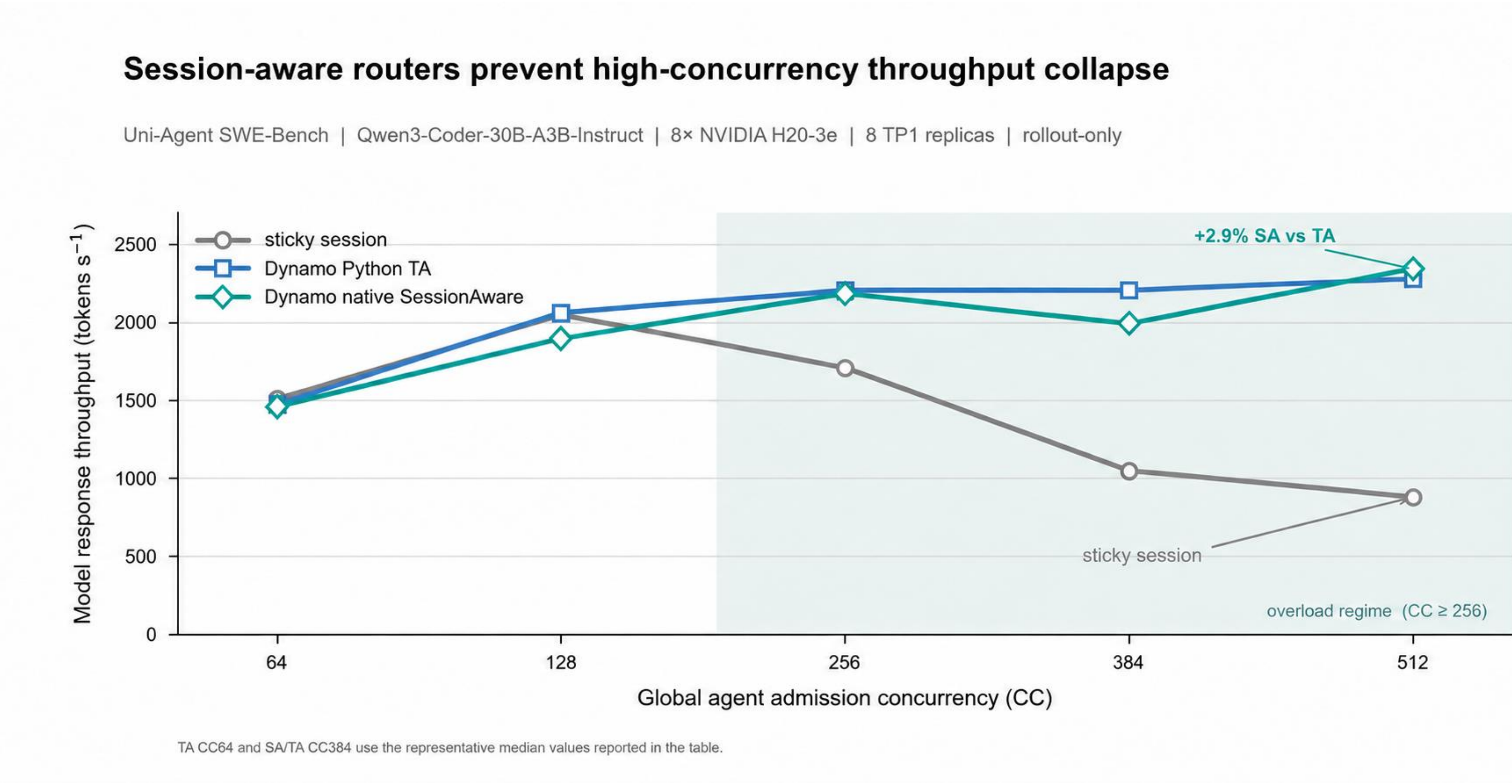
Agentic Serving ThunderAgent Benchmarks

SWE Bench Lite – 300 session replay	No-TA · LRU	No-TA SLRU	Dynamo ThunderAgent
e2e 中位	582s	564s	290s
e2e 均值	847s	837s	409s
TTFT p50	1866ms	1729ms	250ms
TPOT p50	112.5ms	116.3ms	55.9ms
gen 吞吐 中位 (tok/s)	208	119	499
gen 吞吐 均值 (tok/s)	305	241	502
Prefix 命中率	40.2%	40.3%	93.0%
evict 重算批次	70%	70%	4%
usage ≥ 0.95 占比	61%	60%	0%
TA pause 累计 *	N/A	N/A	45
TA resume 累计 *	N/A	N/A	771

pause 替代驱逐:Prefix 命中 40% → 93%,e2e 中位 -50%,TTFT p50 -87%,evict 重算 70% → 4%

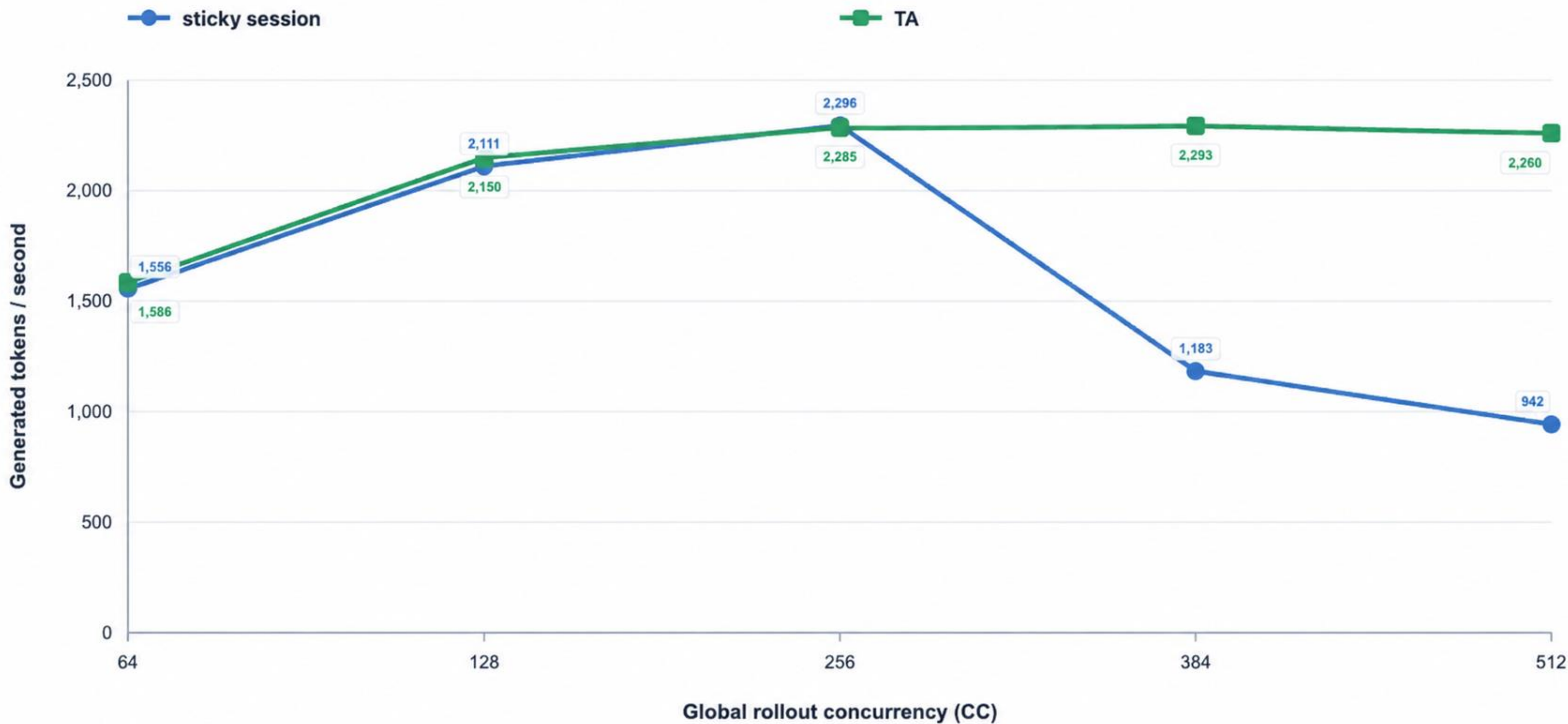
Dynamo Thunder Agent in verl

verl | uni-agent | SWE-bench



E2E RL rollout-phase generated-token throughput (CC64–512)

Exact response-token counts divided by the measured rollout wall time. Values are extracted from the rollout phase of the same end-to-end RL runs.



Tested in this PR: <https://github.com/verl-project/verl-recipe/pull/126>
github.com/verl-project/verl-recipe/tree/main/dynamo

CC384	1.94×	rollout	CC512	2.40×	rollout
	1.39×	full step		1.60×	full step

Native SessionAware routing moves into Dynamo's core

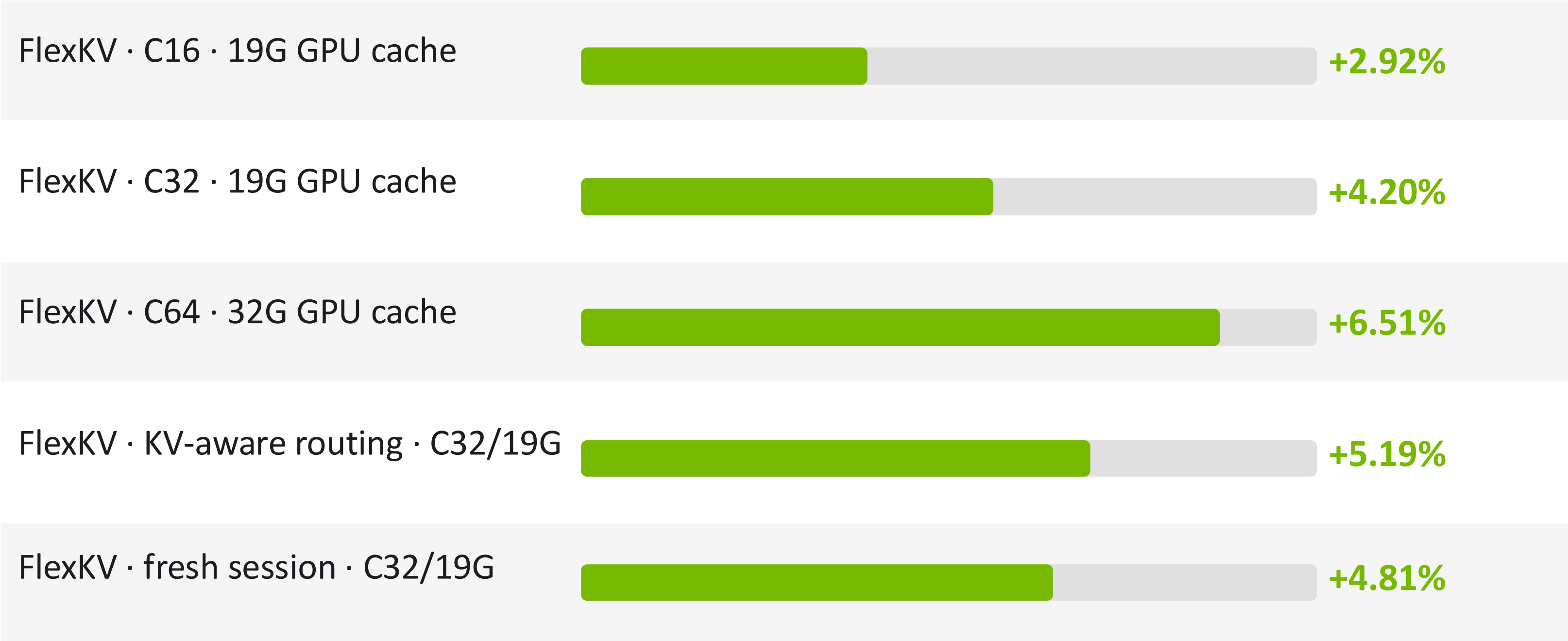
CC	Router	P99 tok/s	Prefix hit	KV mean	Waiting mean*	Preemptions
64	Native SA	1,430.2	95.97%	12.9%	0.00	0
	Python TA	1,647.6	96.02%	12.9%	0.00	0
	Global LB	1,691.4	96.01%	13.0%	0.00	0
128	Native SA	1,899.9	95.90%	27.2%	0.003	0
	Python TA	2,076.7	95.88%	30.3%	0.00	0
	Global LB	2,048.4	95.91%	29.5%	0.00	0
256	Native SA	2,280.5	93.02%	48.0%	0.55	0
	Python TA	2,232.0	93.18%	50.2%	0.27	0
	Global LB	1,701.6	34.46%	72.9%	7.44	40
384	Native SA	2,340.3	91.23%	46.9%	1.01	0
	Python TA	2,399.1	91.51%	52.5%	0.87	0
	Global LB	1,086.3	14.72%	79.9%	85.24	139
512	Native SA	2,325.9	90.90%	53.7%	2.45	0
	Python TA	2,264.9	90.28%	54.1%	4.64	0
	Global LB	902.6	11.40%	80.7%	145.02	145

32B Dense Model: Speedup Charts & Analysis

Qwen3-32B-Instruct-FP8 · 8×H20 · TP2×DP4 · Uni-Agent SWE-agent RL · max_turns=15 · max_model_len=40960

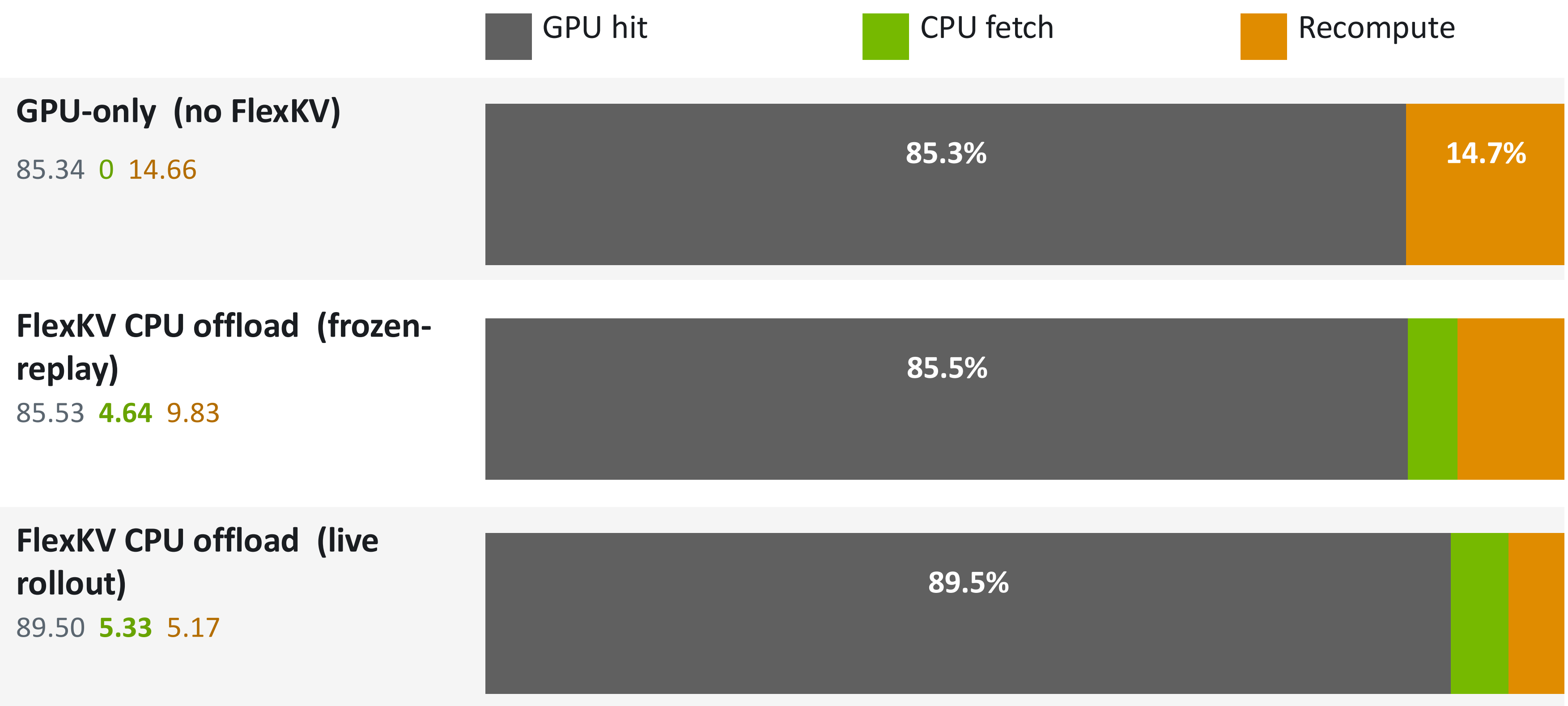
Frozen replay: 64 fixed trajectories (8 SWE prompts × 8 rollouts) replayed at steady-state — stable KV reuse measurement · G = GPU-only baseline (no FlexKV) · teacher-forced: prompt frozen per turn, response tokens discarded
C{n} = concurrent trajectory count (replay stress level) · {n}G/rank = GPU KV cache size (--kv-cache-memory-bytes) · higher concurrency → more simultaneous KV transfers → tighter GPU headroom

Frozen-replay speedup vs GPU-only baseline



Frozen-replay speedup vs GPU-only (no FlexKV); C{n} = GPU KV block reservation (capacity gate)

Prompt-token path breakdown (%)



C32 fixed: FlexKV offload shifts ~4.64% prompt tokens from recompute to CPU fetch (frozen-replay)

Key Insights

Capacity Gate Is the Primary Variable

C16/C32/C64 = concurrent trajectory count. Higher concurrency → more simultaneous KV transfers → GPU cache under heavier load. Validated by offline capacity gate: min observed free GPU blocks must stay ≥ P95 transfer payload size. Speedup climbs 2.92%→4.20%→6.51% with concurrency because more concurrent trajectories expose more cross-session KV reuse.

CPU Fetch Replaces Recompute, Not GPU Hits

GPU hit rate stays nearly flat (85.34% → 85.53%); Recompute drops 14.66% → 9.83%, with the 4.83% gap fully captured by CPU fetch. FlexKV acts on the GPU-miss tail, leaving GPU cache behavior unchanged.

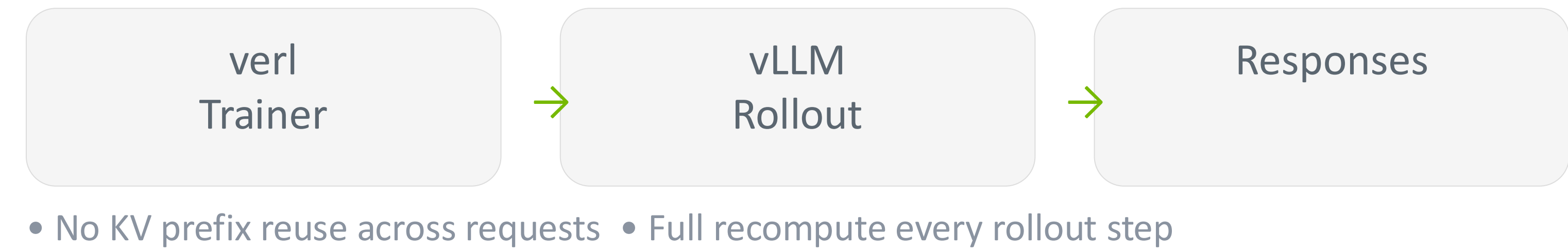
Dynamic Routing Adds Extra +1%

Same C32/19G config: KV-aware routing raises CPU fetch from 4.64% to 5.93%. Router steers each request to the shard with existing prefix, reducing cache misses.

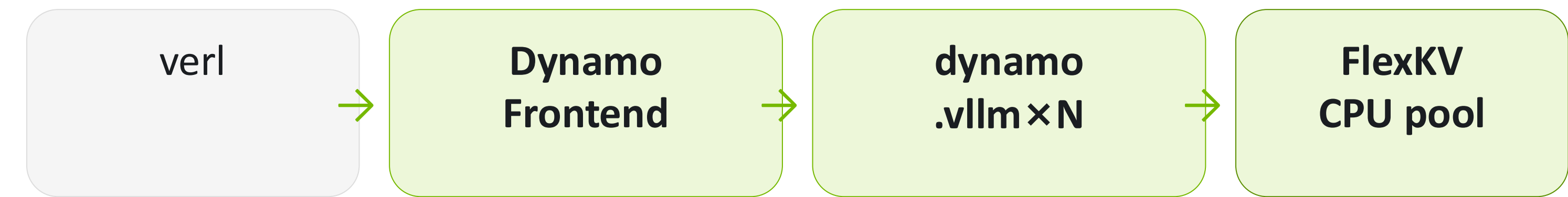
FlexKV Integration Delta

5 targeted changes on top of baseline verl + vLLM rollout — zero verl-core modification

Baseline (verl + plain vLLM rollout)



After (verl + Dynamo + vLLM + FlexKV)



5 incremental changes

recipe.dynamo.register **DynamoLLMStateManager**

Module loaded via VERL_USE_EXTERNAL_MODULES=recipe.dynamo.register; import side-effect patches LLMStateManager → full Dynamo lifecycle management

--worker-extension-cls **vLLMDynamoColocateWorkerExtension**

injects ZMQ IPC socket path fix for colocated mode (VERL_DYNAMO_RANK_OFFSET)

DYNAMO_USE_FLEXKV=1 **FlexKV CPU KV pool**

enables CPU-side KV block server (get_match / put_match)

FLEXKV_SHARED_CPU_CACHE=1 **shared pool across all DP groups**

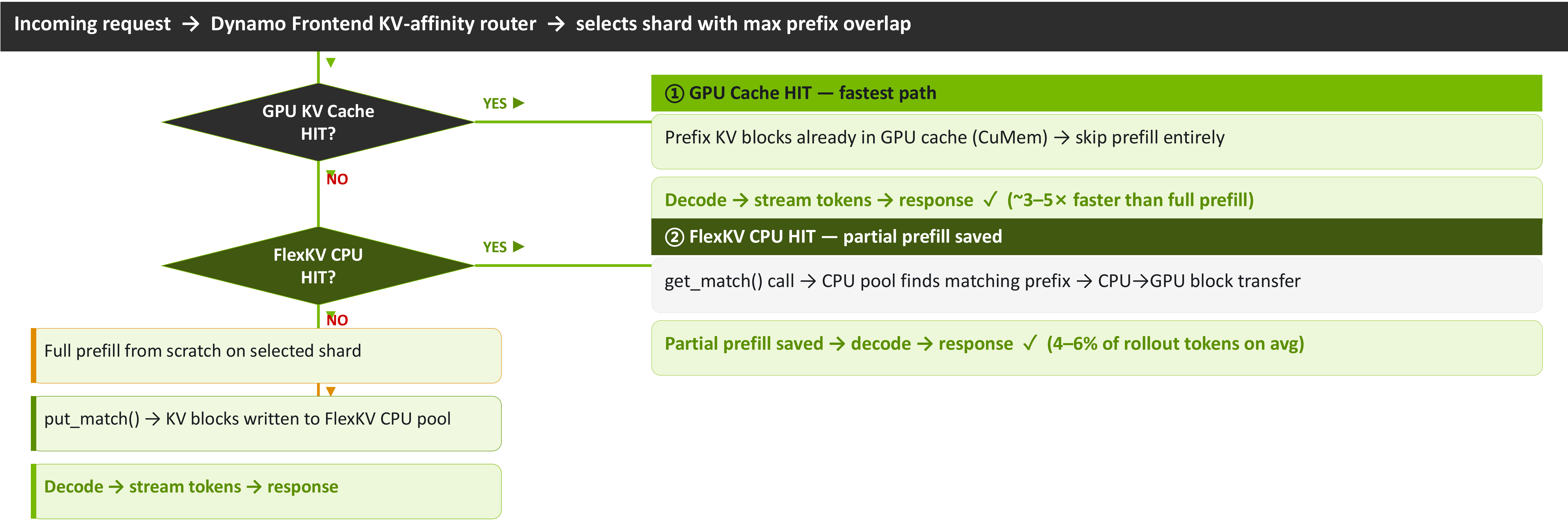
cross-DP KV visibility — accounts for 99.5% of CPU fetch benefit

router_mode: kv **KV-affinity routing**

YAML config rollout.engine_kwargs.dynamo.router_mode · routes each request to shard with max prefix hash overlap

KV Cache Flow

Each request follows one of three paths — GPU hit (fastest) · CPU hit · cache miss



⚠️ RL Correctness Boundary — Weight Update

`clear_kv_cache(reset_connector=True)` → GPU prefix cache + FlexKV CPU blocks INVALIDATED after every weight update

KV blocks encoded by old policy weights must not be reused by new policy — `clear_kv_cache` enforces this invariant

How to Integrate

A drop-in rollout backend — registered from recipe, zero verl-core change

1

Register

recipe/dynamo/register.py

```
# No register() function – module import is
the side-effect
# Triggered at startup via
VERL_USE_EXTERNAL_MODULES
import verl.recipe.dynamo.register
# → LLMServerManager patched to
DynamoLLMServerManager
# → DynamoReplica registered for rollout
```

2

Configure

config/dynamo_trainer.yaml

```
rollout:
  name: dynamo
  mode: async
  clear_kv_cache_after_update: true
  engine_kwargs.dynamo:
    router_mode: kv
    enable_rl: true
```

3

Launch

env vars + main_dynamo.py

```
export
VERL_USE_EXTERNAL_MODULES=recipe.dynamo.register
python -m recipe.dynamo.main_dynamo ...ppo
actor_rollout_ref.rollout.name=dynamo \
++actor_rollout_ref.rollout.engine_kwargs.dynamo.
router_mode=kv \
++actor_rollout_ref.rollout.engine_kwargs.dynamo.
return_tokens_as_token_ids=false \
++actor_rollout_ref.rollout.engine_kwargs.dynamo.
request_engine_data=true \
++actor_rollout_ref.rollout.engine_kwargs.dynamo.
request_completion_token_ids=true \
++actor_rollout_ref.rollout.engine_kwargs.dynamo.
thunderagent.enabled=${thunderagent_enabled}
```

Everything ships inside recipe/dynamo/ (recipe-only, zero verl-core change)

verl core is untouched — the recipe registers the backend via VERL_USE_EXTERNAL_MODULES at import time. FlexKV CPU pool is opt-in via DYNAMO_USE_FLEXKV=1; clear_kv_cache is automatic when clear_kv_cache_after_update=True.

